

Latin Modern Roman

Alert & Notification Microservice

Proposal for a Standalone Alert Service in the EMIS Ecosystem

EMIS Development Team

19 June 2026

Branch: `be/feat/alert-service`

Status: Proposed

Executive Summary

The Alert page (the officer “Inbox”) is currently fetched directly from the Laravel backend, mixing alert data with the rest of the HRM domain and giving future services no clean way to raise notifications to officers. This proposal recommends building a dedicated, standalone **Alert and Notification Microservice** in Node.js + Express with its own MySQL database.

The service is designed from day one to evolve from a polling-based data source (Phase 1) into a fully event-driven publish model (Phase 2) without any change to the frontend contract. It incorporates a hardened, layered security architecture covering JWT validation, rate limiting, security headers, input sanitization, and database access controls.

The upstream dependency (currently the Laravel backend directly; WSO2 APIM in approximately one month) is abstracted behind a single environment variable so the transition requires no code change.

Contents

1	Background	4
2	The Problem	4
2.1	Alert data is coupled to the main Laravel backend	4
2.2	This does not scale	4
3	The Proposal	4
3.1	What it is	4
3.2	What it is not	5
4	Main Purpose	5
5	Current Infrastructure Note	5
6	System Architecture	5
6.1	Why a microservice	5
6.2	System context — current state (no APIM)	6
6.3	System context — future state (with APIM in ~1 month)	6
6.4	Service responsibilities	6
7	Two-Phase Design	7
7.1	Phase 1 — Foundation (current scope)	7
7.2	Phase 2 — Event-driven (future scope)	8
8	Security Architecture	8
8.1	Middleware execution order	8
8.2	Transport security	8
8.3	Security headers (Helmet)	8
8.4	CORS	8
8.5	Rate limiting	9
8.6	JWT validation (user auth)	9
8.7	Service-to-service auth (Phase 2)	10
8.8	Input validation	10
8.9	Database security	10
8.10	Error handling — no information leakage	10
8.11	Request logging	11
8.12	Dependency security	11
8.13	Container security	11
9	Project Structure	11
10	Technology Stack	12
11	R&D: Database Technology Selection	12
11.1	Evaluation constraints	12
11.2	Why NoSQL is a strong candidate	13
11.3	Candidate evaluation	13

11.4 Recommendation	14
11.5 Why not MySQL for this service	15
12 Database Design	15
12.1 alerts collection	15
12.2 alertcounts collection	15
12.3 servicekeys collection	16
13 API Surface (Frontend Contract)	16
14 Data Flow	16
14.1 Counts request (Primary tab)	16
14.2 List request (any tab)	17
15 Environment Configuration	17
16 Impact on the Frontend Alert Page	17
17 What This Enables in the Future	18
18 Deployment	18
18.1 Local development	18
18.2 Docker (local stack)	19
18.3 Production	19
18.4 Pre-production security checklist	19
19 Summary	19

1 Background

The EMIS system manages the appointment and verification workflow for teachers and principals across schools in Sri Lanka. When a profile is submitted it passes through a structured approval chain involving verification officers, zonal education officers, and employer representatives.

The Alert page is currently the only place in the system where officers can see what needs their attention. It surfaces four categories of work:

- Profiles waiting for first-level verification
- Profiles that were rejected and subsequently revised by the applicant
- Profiles waiting for employer confirmation
- Profiles that have been fully rejected

This page is critical to daily operations. Officers log in and check it to know what actions they owe. Without it, the approval workflow has no visibility layer.

2 The Problem

2.1 Alert data is coupled to the main Laravel backend

- **Alert data lives inside Laravel’s database**, mixed with teacher profiles, institutions, timetables, and everything else. There is no domain separation.
- **Future services cannot contribute to the Alert page.** A future payroll or transfer-request service has no clean way to surface notifications to officers without writing directly into Laravel’s database, defeating the purpose of having separate services.
- **The frontend code is inconsistent.** Alert data is currently fetched in three different ways across five different component files with no single source of truth.
- **No centralized security.** Each component calls the backend independently with no shared rate limiting, input validation, or audit logging for alert traffic.

2.2 This does not scale

As EMIS grows, each new service will generate its own events that officers need to see. Without a dedicated alert service, every new service must couple to Laravel or the frontend must call multiple backends to assemble a single notification view.

3 The Proposal

We propose building a **dedicated Alert and Notification Microservice**: a standalone Node.js + Express backend with its own MySQL database and a hardened security stack.

3.1 What it is

- A standalone HTTP service that owns all alert and notification data in its own database
- The service that powers the Alert page (Inbox) in the EMIS frontend

- The common ground for alerts across all current and future EMIS microservices
- A security-hardened service with layered defences (see Section 8)

3.2 What it is not

- Not a replacement for the main Laravel backend
- Not a messaging system — no email or SMS in Phase 1
- Not a real-time push system — no WebSockets in Phase 1
- Not a pass-through proxy — it owns its own database from day one

4 Main Purpose

Core purpose

The alert service is the single place in the EMIS ecosystem where any service can publish an alert, and any authorized user can read what needs their attention.

Today that means teacher and principal appointment workflow alerts for HR officers. Tomorrow that means any event from any service — a payroll discrepancy, a transfer request, a document expiry — flows through the same service and appears in the same officer inbox.

The service gives EMIS a **common notification layer** independent of any single backend technology. A Python service, a Go service, or a second Node service can all speak to one place.

5 Current Infrastructure Note

Important: APIM not yet in production

WSO2 API Manager (APIM) is **not currently deployed** in the EMIS environment. It is expected to be available in approximately one month. This affects the upstream URL the alert service calls in Phase 1.

The upstream dependency is abstracted behind a single environment variable `UPSTREAM_BASE_URL`. Today it points directly to the Laravel backend; when APIM is deployed, updating this one variable and redeploying is all that is required. No code changes are needed.

Period	UPSTREAM_BASE_URL value
Now (no APIM)	<code>https://backend.app-uat.emis.moe.gov.lk/api</code>
In ~1 month (with APIM)	<code>https://services.app-uat.emis.moe.gov.lk/hrm/1.0.0</code>

6 System Architecture

6.1 Why a microservice

The alert service is standalone — not a module inside Laravel, not a proxy wrapper — because it **owns its own database**. That is what makes it independently deployable and the common ground for all future EMIS services.

Problem with current setup	What the microservice solves
Alert data lives in Laravel's DB, coupled to the HRM domain	Alert service owns its own DB, fully decoupled
Future services cannot publish alerts without touching Laravel	Any service can publish to the alert service's API
Frontend fetches alerts from different sources inconsistently	One service, one contract, one URL
Cannot scale alert reads independently	Alert service scales on its own
No centralized security layer for alert traffic	Hardened middleware stack on every request

6.2 System context — current state (no APIM)

Browser (React SPA)

Alert page → alertService.js → Alert Microservice URL

↓ HTTPS Authorization: Bearer <token>

Alert Microservice (Node.js + Express)

Security stack: Helmet → CORS → Rate Limit → JWT Validate → Input Validate

Owns: Alert MySQL DB (alerts, alert_counts, service_keys)

Phase 1: sync middleware reads from Laravel directly

↓ HTTPS Authorization: Bearer <token>

Laravel Backend (FrankenPHP)

JwtGuard validates token → returns alert data from its own DB

6.3 System context — future state (with APIM in ~1 month)

Browser (React SPA)

↓

Alert Microservice (Node.js + Express)

↓ HTTPS Authorization: Bearer <token>

WSO2 API Manager (APIM)

Validates token → strips /hrm/1.0.0 → forwards to Laravel

↓ HTTP

Laravel Backend (FrankenPHP)

JwtGuard validates token → returns alert data from its own DB

Transition

Switching from the current state to the APIM state requires changing `UPSTREAM_BASE_URL` in the environment and redeploying. Zero code changes.

6.4 Service responsibilities

In scope (Phase 1):

- Own and manage the alert MySQL database
- Expose a hardened HTTP API for reading alert data (counts + paginated lists)
- Validate every incoming JWT independently — not relying solely on the upstream backend

- Rate-limit all endpoints to protect upstream services and the DB
- Validate and sanitize all input before it reaches the DB or upstream calls
- Phase 1: populate the DB by reading from the upstream backend via an internal sync middleware
- Normalize all data into a consistent camelCase shape before storing and serving
- Return consistent, information-safe error responses — no stack traces, no internal details

Out of scope (Phase 1):

- Accepting alert events published by producers (Phase 2)
- WebSocket / Server-Sent Events for real-time updates (future)
- Performing workflow actions (verify, confirm, reject) — these remain direct frontend-to-backend calls
- Email or SMS notifications (future)

7 Two-Phase Design

The service is designed so that moving from Phase 1 to Phase 2 only replaces the data-source layer. The DB schema, controllers, security stack, and the API the frontend calls are identical in both phases.

7.1 Phase 1 — Foundation (current scope)

Goal: get the Alert page reading from the microservice's own database instead of the backend directly.

Component	Description
Node.js + Express service	Standalone HTTP server at its own URL
Alert MySQL database	Own database: <code>alerts</code> , <code>alert_counts</code> , <code>service_keys</code>
Sync middleware	Fetches from <code>UPSTREAM_BASE_URL</code> /Laravel, upserts into alert DB
Security stack	Helmet, CORS, rate limiter, JWT validation, input validation
REST API	5 endpoints: counts + 4 paginated lists
Frontend service file	<code>alertService.js</code> — the only frontend file that knows the microservice URL

What the frontend Alert page gains:

- One clean URL for all alert data
- Consistent camelCase response shapes
- Rejection reasons included directly in the rejected list — no secondary API calls
- A security-hardened access layer for alert traffic
- A foundation ready for Phase 2 with no frontend changes required

7.2 Phase 2 — Event-driven (future scope)

Goal: remove the polling dependency on the upstream backend; producers publish alert events directly to the service.

What	Phase 1	Phase 2
Data source	Sync middleware polls upstream	Producers POST events here
Laravel role	Source of data	Publisher of events
New services	Cannot contribute alerts	Publish with a service API key
DB schema	Designed for Phase 2	Unchanged
Frontend API	—	Unchanged
Security	User JWT + CORS + rate limit	Same + API key auth for producers

The DB schema is designed for Phase 2 from day one so no migration is needed at the switch.

8 Security Architecture

Defense in depth

Every request passes through a layered security stack. A request that fails any layer is rejected immediately and never reaches the next layer.

8.1 Middleware execution order

1. **Helmet** — set security headers on every response (before anything else)
2. **CORS** — reject requests from unknown origins
3. **Rate Limiter** — reject if request rate exceeds threshold
4. **Auth / Service Auth** — validate JWT (user routes) or API key (producer routes)
5. **Input Validation** — reject malformed or out-of-range parameters
6. **Controller** — business logic, calls service layer
7. **Error Handler** — catch all thrown errors, return safe response

8.2 Transport security

All traffic in production goes over **HTTPS only**. TLS termination is handled at the infrastructure layer (nginx-proxy or load balancer), consistent with the existing EMIS stack. The Express app runs HTTP behind the TLS terminator.

8.3 Security headers (Helmet)

helmet is applied as the **first middleware** — before CORS, auth, or anything else. It sets protective HTTP response headers on every response.

8.4 CORS

Strict, explicit policy — no wildcard.

Header	Value	Purpose
X-Content-Type-Options	nosniff	Prevent MIME-type sniffing
X-Frame-Options	DENY	Prevent clickjacking
Strict-Transport-Security	max-age=31536000	Enforce HTTPS for 1 year
Content-Security-Policy	default-src 'none'	Block all content embedding
X-Powered-By	(removed)	Do not expose the framework name
Referrer-Policy	no-referrer	Do not leak URL in Referer header

Setting	Value
origin	CORS_ORIGIN env var only (exact frontend URL)
methods	GET, OPTIONS (user routes only — no POST/DELETE from browser)
allowedHeaders	Authorization, Content-Type only
credentials	true

8.5 Rate limiting

Three tiers of rate limiting using `express-rate-limit`, applied per IP address. Rate limit headers are returned in every response so clients can self-throttle.

Tier	Applied to	Window	Max requests	On breach
Global	All routes	15 min	500	429 TOO_MANY_REQUESTS
Alert reads	GET /api/alerts/*	1 min	60	429 TOO_MANY_REQUESTS
Health check	GET /health	1 min	120	429 TOO_MANY_REQUESTS

8.6 JWT validation (user auth)

Key change from original design

The alert service **validates JWTs itself** using the WSO2 IS JWKS endpoint. It does not rely solely on the upstream backend to be the gatekeeper. A request with a fake, expired, or malformed token is rejected before any DB or upstream sync call is made.

Token validation now occurs at **three points**:

1. **Alert Service** (new) — signature + claims verified before anything else happens
2. **APIM** (when deployed) — full OAuth token introspection
3. **Laravel backend** — JwtGuard JWKS validation

The `auth` middleware:

1. Extracts the Bearer token. Missing or malformed $\Rightarrow$ 401 immediately.
2. Fetches the JWKS from `WSO2_JWKS_URI` (cached in memory for `JWKS_CACHE_TTL` seconds, default 3600 — matching Laravel's behaviour).
3. Verifies the token signature using the matching public key.
4. Verifies `exp` (not expired), `iss` (matches `WSO2_ISSUER`), and `aud` (if `WSO2_AUDIENCE` is set).

5. If any check fails $\Rightarrow$ 401 immediately. No sync call is made. No DB is touched.
6. If all checks pass $\Rightarrow$ decoded payload attached to `req.jwtPayload`.

Libraries: `jsonwebtoken` for verification; `jwt-key` for JWKS fetching and automatic key rotation handling.

8.7 Service-to-service auth (Phase 2)

When producers publish alert events they use a dedicated **API key**, not a user JWT.

- Header: `X-Service-Key: <api-key>`
- Keys are generated as 64-character cryptographically random hex strings
- Only the **SHA-256 hash** of the key is stored in the `service_keys` DB table — the plain-text key is never stored anywhere
- A compromised key is revoked by setting `active = 0` in the table — no redeploy needed
- This middleware is applied **only** to producer routes (`POST /api/events/*`); user-facing read routes never see it

8.8 Input validation

All query parameters are validated and sanitized using `express-validator` before the controller is reached. A request with invalid parameters is rejected with 400 `BAD_REQUEST`.

Parameter	Rule	On failure
<code>page</code>	Integer, min 1, max 10000	400 <code>BAD_REQUEST</code>
<code>per_page</code>	Integer, min 1, max 100	400 <code>BAD_REQUEST</code>

Extra query parameters not in the allowed list are stripped silently before the request reaches the controller.

8.9 Database security

- **Restricted DB user:** `DB_USER` is granted only `SELECT`, `INSERT`, `UPDATE` on the alert tables. No `DROP`, no `GRANT`. A separate `DB_MIGRATE_USER` with broader privileges is used only at startup for migrations.
- **Parameterized queries only:** all DB queries use `mysql2` parameterized syntax. String concatenation into SQL is never used. This fully prevents SQL injection.
- **Sensitive data:** the `payload JSON` column stores only display data (names, school, appointment dates). No NIC numbers, addresses, or credentials.

8.10 Error handling — no information leakage

- In **production**, all errors return a generic message to the client. No stack traces, no DB query text, no internal details.
- Full error detail (message + stack + request ID) is written to server logs only.

- Upstream errors (backend, future APIM) are translated — the client sees `UPSTREAM_ERROR`, not the raw upstream response body.
- Error responses never include the values of request headers or query parameters.

Consistent error response shape:

```
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Token validation failed." } }
```

8.11 Request logging

The `requestLogger` middleware logs request ID, method, path, status code, response time, and client IP. It **never** logs the value of the `Authorization` header, the `X-Service-Key` header, any part of the JWT payload, or any request/response body.

8.12 Dependency security

- `package-lock.json` is committed — exact versions are pinned
- `npm audit` runs in CI/CD on every push; builds fail on high-severity findings
- No new dependency is added without reviewing its CVE history

8.13 Container security

- Runs as a **non-root user** (UID 1000) inside the container
- **Multi-stage Docker build** — only production dependencies in the final image, no dev tools, no build artifacts
- Uses the official `node:20-alpine` base image — minimal attack surface
- `.dockerignore` excludes `.env`, `node_modules`, test files, and documentation

9 Project Structure

The service lives at the repo root as a peer of `backend/` and `frontend/`:

```
emis/
  backend/           Laravel + FrankenPHP (existing)
  frontend/          React + Vite (existing)
  alert-service/      Node.js + Express + MySQL (NEW)
  infrastructure/
  docs/
```

Inside `alert-service/src/`:

```
src/
  app.js             Express app setup, middleware registration in order
  server.js           HTTP server bootstrap
  config/index.js     Validate env vars at startup; crash on missing required
vars
  routes/
    alertRoutes.js    GET /api/alerts/* (user-facing)
```

eventRoutes.js	POST /api/events/* (Phase 2 producer routes, stubbed)
controllers/ alertController.js	Extract validated params → call service → respond
services/ alertService.js	Query alert DB, return shaped data
syncService.js	Fetch from upstream, upsert into alert DB
db/ connection.js	mysql2 connection pool
migrations/ 001_create_alerts.sql	
002_create_alert_counts.sql	
003_create_service_keys.sql	
middleware/ helmet.js	Security headers (first middleware registered)
cors.js	CORS --- locked to CORS.ORIGIN
rateLimiter.js	Three-tier rate limiting
auth.js	Full JWT validation (signature + claims)
serviceAuth.js	Phase 2 API key validation for producers
validate.js	Input sanitization (express-validator)
errorHandler.js	Centralised safe error responses
requestLogger.js	Structured logging, never logs tokens or keys

Layer	File	Responsibility
Security	helmet.js	Set HTTP security headers on every response
Security	cors.js	Reject requests from unknown origins
Security	rateLimiter.js	Three-tier rate limiting per IP
Auth	auth.js	Validate JWT signature + claims (JWKS-based)
Auth	serviceAuth.js	Validate producer API key (Phase 2)
Validation	validate.js	Sanitize and validate all query parameters
Route	alertRoutes.js	Map HTTP method + path to controller
Controller	alertController.js	Extract params, trigger sync, call service, respond
Service	alertService.js	Query the alert DB, return shaped data
Sync	syncService.js	Fetch from upstream, normalize, upsert into DB
DB	db/connection.js	Manage mysql2 connection pool
Error	errorHandler.js	Catch all errors; safe response; full detail in logs
Logging	requestLogger.js	Structured request logging (no secrets)
Config	config/index.js	Validate env vars at startup; fail fast on missing

10 Technology Stack

11 R&D: Database Technology Selection

Before committing to a database engine, an evaluation was conducted to determine whether a relational (SQL) or non-relational (NoSQL) database better fits the alert service. This section documents the constraints, findings, and recommendation from that evaluation.

11.1 Evaluation constraints

Two hard constraints were identified for any database used in this service:

- **Free and open source.** The service must not introduce a paid database licence.

Concern	Technology	Reason
Runtime	Node.js 20 LTS	Lightweight, fast non-blocking I/O
Framework	Express 4	Minimal, well-understood, no magic
Document store	MongoDB Community	Self-hosted; flexible document model for variable alert payloads; no overseas dependency
DB ODM	mongoose	Schema definition, validation, and query abstraction for MongoDB
Event queue (Phase 2)	Redis	Self-hosted; Redis Streams for durable async event delivery from producers
HTTP client	axios	Same library used in the frontend
JWT validation	jsonwebtoken + jwks-rsa	Industry standard; same approach as Laravel's <code>firebase/php-jwt</code>
Security headers	helmet	One dependency, covers all OWASP header recommendations
Rate limiting	express-rate-limit	Lightweight, no external dependency
Input validation	express-validator	Declarative, well-maintained
Environment	dotenv	Standard; <code>.env.example</code> pattern already used across this repo
Logging	morgan + structured console	No external log service dependency in Phase 1
Containerisation	Docker (multi-stage, non-root)	Consistent with <code>backend/</code> and <code>frontend/</code>

- **Self-hosted only — no overseas location.** As part of the EMIS system operated by the Ministry of Education, Sri Lanka, all data must remain on infrastructure inside the country. This is a data sovereignty requirement. It rules out all managed cloud database services (MongoDB Atlas, AWS DynamoDB, AWS DocumentDB, Azure Cosmos DB, Google Firestore, etc.) regardless of their technical merits — they host data on overseas servers.

11.2 Why NoSQL is a strong candidate

Alert and notification data has characteristics that favour a document-oriented store:

- **Variable payload per producer.** Each producer service may include different fields in its alert payload. A document model stores these naturally without requiring schema migrations when a new producer is added.
- **Messaging and task handling patterns.** The intended use of this service — collecting events from multiple producers and delivering them to role-based inboxes — is a pattern where NoSQL document and key-value stores are widely used in production.
- **No complex joins required.** Alert queries are simple: filter by type, status, and role. There are no multi-table joins. A relational schema gives no structural advantage.
- **Future event volume.** As more producer services come online in Phase 2, the volume of alert writes will grow. Document stores generally scale write throughput more cheaply than relational engines.

11.3 Candidate evaluation

Only free, self-hostable engines were considered.

Engine	Type	Free	Strengths for this use case	Complexity
MongoDB Community	Document	Yes	Flexible document model; native JSON queries; well-known driver ecosystem; runs easily in Docker; straightforward for alert storage with variable payloads	Low
Redis	Key-value + Streams	Yes	Industry standard for queuing, pub/sub, and task queues; Redis Streams maps directly to Phase 2 event publishing; very low latency	Low
Apache Cassandra	Column-family	Yes	Excellent for high-throughput time-series events; linear horizontal scale	High (requires cluster)
Apache CouchDB	Document	Yes	Document model; HTTP-native API	Low
MySQL 8	Relational	Yes	Already running in EMIS; native JSON column covers flexible payload; ACID transactions; familiar to the ops team	Low

11.4 Recommendation

Selected engines

MongoDB Community for persistent alert document storage.

Redis for Phase 2 async event queuing and pub/sub (when producers publish directly).

MongoDB Community is recommended for the alert document store because:

- The alert record is naturally a document — envelope fields (**alertType**, **status**, **peopleId**) are fixed; payload content varies per producer. MongoDB stores both without a JSON-column workaround.
- Self-hosted in Docker alongside the other EMIS services — no overseas dependency, no licence cost. Replaces the **alerts** and **alert_counts** MySQL tables with a single **alerts** collection.
- Simple to operate at EMIS's current scale. A single replica-set node is sufficient for Phase 1 and can be expanded without application changes when needed.

Redis is recommended as a **Phase 2 addition** for the event queue when producers begin publishing alert events asynchronously. Redis Streams provides durable, ordered event delivery with consumer groups — suited to the producer/consumer model in Phase 2. Redis does not replace MongoDB; it sits in front of it as the intake queue.

11.5 Why not MySQL for this service

MySQL is already proven in EMIS and is not a wrong choice. The deciding factors for moving to MongoDB here are:

- The **payload** JSON workaround in MySQL is functional but turns the DB into a hybrid that is neither fully relational nor fully document-oriented.
- Adding new producer alert types in Phase 2 requires no schema changes in MongoDB — a new producer simply adds new fields to its payload document.
- Keeping the alert service on its own NoSQL engine reinforces its independence from the main EMIS MySQL instance and makes the service easier to relocate or replicate separately.

12 Database Design

The MongoDB database uses three collections. The schema is designed generically so any future producer can contribute alerts without a migration.

Driver: `mongoose` (ODM for Node.js + MongoDB). Replaces `mysql2` in the technology stack.

12.1 alerts collection

One document per alert record. All producer-specific display data lives inside the **payload** sub-document — its shape varies per producer and requires no schema change when a new producer type is added.

Field	Description
<code>_id</code>	MongoDB ObjectId (auto-generated)
<code>alertType</code>	Category: <code>pending_verification</code> , <code>revised</code> , <code>pending_confirmation</code> , <code>rejected</code>
<code>producer</code>	Which system created this alert. Phase 1: <code>emis-hrm</code> . Future: any service name
<code>peopleId</code>	The teacher or principal this alert relates to
<code>recipientRole</code>	Which role should see this alert; <code>null</code> = all authorized roles
<code>status</code>	<code>active</code> = needs attention, <code>resolved</code> = actioned, <code>archived</code> = hidden
<code>payload</code>	Sub-document — all display data (name, school, appointment date, rejection reason). Shape varies per producer; no schema constraint.
<code>externalRef</code>	ID in the producer's system; used for upsert deduplication during Phase 1 sync
<code>syncedAt</code>	Last time this record was refreshed from upstream (Phase 1 only)
<code>createdAt</code>	Document creation timestamp (auto-managed by Mongoose timestamps)

Indexes: `alertType`, `peopleId`, `status`, `syncedAt`. Unique compound index: `{ alertType, externalRef }` for safe upsert deduplication.

12.2 alertcounts collection

One document per alert category, updated on every sync cycle. Powers the overview tab cards and tab badge numbers. The `alertType` field is the unique key — at most four documents in this

collection.

12.3 servicekeys collection

Stores hashed API keys for Phase 2 producers. Only the SHA-256 hash is stored; the plain-text key is never written to the database.

Field	Description
<code>_id</code>	MongoDB ObjectId
<code>serviceName</code>	Human-readable producer service name (e.g. <code>emis-hrm</code>)
<code>keyHash</code>	SHA-256 hash (hex, 64 chars) of the API key — never the plain-text key
<code>active</code>	<code>true</code> = valid, <code>false</code> = revoked. Revoke without deleting for audit trail
<code>createdAt</code>	Key creation timestamp
<code>lastUsedAt</code>	Last successful authentication timestamp

13 API Surface (Frontend Contract)

The frontend API contract is **identical between Phase 1 and Phase 2** — only the internal data source changes.

Endpoint	Purpose
GET <code>/health</code>	Public health check — no auth; used by Docker health checks
GET <code>/api/alerts/counts</code>	Count per alert category; powers overview cards and tab badges
GET <code>/api/alerts/pending-verification</code>	Paginated list of profiles awaiting first-level verification
GET <code>/api/alerts/revised</code>	Paginated list of profiles revised and resubmitted after rejection
GET <code>/api/alerts/pending-confirmation</code>	Paginated list of profiles awaiting employer confirmation
GET <code>/api/alerts/rejected</code>	Paginated list of rejected profiles, including rejection reason

Request convention: all protected endpoints require `Authorization: Bearer <token>`. Missing or invalid token $\Rightarrow$ 401 immediately.

Response conventions:

- Success: `{ "success": true, "data": <payload> }`
- Error: `{ "success": false, "error": { "code": "...", "message": "..." } }`
- All property names are **camelCase** — upstream snake_case is normalized before storage
- Pagination via `page` and `per_page` query parameters (max `per_page`: 100)

14 Data Flow

14.1 Counts request (Primary tab)

1. React `Alert.jsx` mounts; `alertService.getAlertCounts()` is called
2. GET `/api/alerts/counts` with `Authorization: Bearer <token>`

3. Helmet applies security headers
4. CORS check passes (frontend origin is allowed)
5. Rate limiter: within limit, continue
6. `auth` middleware: extract Bearer token → fetch JWKS (from cache) → verify signature, `exp`, `iss` → valid; attach payload to request
7. `alertController.getCounts()` called
8. `syncService.syncCounts(token)`: calls `UPSTREAM_BASE_URL/alerts/counts` with Bearer token → upstream validates and returns counts → upserted into `alert_counts` table
9. `alertService.getCounts()`: `SELECT` from `alert_counts`
10. Controller sends 200 with normalized `{ pendingVerification, revised, ... }`
11. React renders count cards

14.2 List request (any tab)

1. User clicks a tab; the list component calls, e.g., `alertService.getPendingVerification({ page: 1, perPage: 20 })`
2. `GET /api/alerts/pending-verification?page=1&per_page=20` + Bearer token
3. Helmet → CORS → rate limiter → JWT validation (all pass)
4. `validate` middleware: `page=1` valid; `per_page=20` valid
5. `syncService.syncList(type, token, page, perPage)`: fetches paginated list from upstream → normalizes → upserts into `alerts` table
6. `alertService.getList(type, page, perPage)`: parameterized `SELECT` from `alerts` with `LIMIT/OFFSET`
7. Controller sends 200 with `{ items: [...], pagination: {...} }`
8. Component sets state; passes data as props to `AlertTeacherList`

15 Environment Configuration

16 Impact on the Frontend Alert Page

The Alert page (Inbox) continues to look and work exactly the same for users. Under the hood:

- Each component now owns its data fetching and passes data down as props — clean separation of data and display
- All alert API calls go through a new `alertService.js` file — changing the data source in the future requires editing one file only
- `AlertTeacherList` becomes a pure display component — no fetch logic, easier to test and reuse

Variable	Required	Description
PORT	No (3001)	Port the Express server listens on
NODE_ENV	No (development)	Controls error detail in responses
UPSTREAM_BASE_URL	Yes	Backend URL (Laravel direct now; APIM in ~1 month)
CORS_ORIGIN	Yes	Exact frontend origin; never a wildcard
WSO2_JWKS_URI	Yes	JWKS endpoint on WSO2 IS / Asgardeo
WSO2_ISSUER	Yes	Expected <code>iss</code> claim in JWT
WSO2_AUDIENCE	No	Expected <code>aud</code> claim; if set, tokens without it are rejected
JWKS_CACHE_TTL	No (3600)	Seconds to cache JWKS public keys in memory
DB_HOST	Yes	MySQL host
DB_PORT	No (3306)	MySQL port
DB_NAME	Yes	Alert service database name (e.g. <code>emis_alerts</code>)
DB_USER	Yes	Restricted MySQL user: SELECT, INSERT, UPDATE only
DB_PASSWORD	Yes	MySQL password
DB_MIGRATE_USER	Yes	Migration user with broader privileges; used at startup only
DB_MIGRATE_PASSWORD	Yes	Migration user password

- Rejection reason data is included directly in the list response from the microservice — the existing secondary API call to fetch rejection comments is eliminated

No visible change to users. Cleaner, safer code for developers.

17 What This Enables in the Future

Future capability	What it requires
Any new EMIS service sends alerts	Add <code>POST /api/events/alert</code> with service API key auth
Real-time count updates	Add WebSocket / Server-Sent Events layer on top of existing DB
Email or SMS notifications	Alert service triggers on new alert records
Notification history / audit trail	Already in the DB from day one — expose a history endpoint
Mobile app inbox	Same REST API, same contract, same auth
APIM integration	Change one env var (<code>UPSTREAM_BASE_URL</code>), redeploy

None of these require touching the main Laravel backend or the frontend Alert page structure.

18 Deployment

18.1 Local development

```
cd alert-service
cp .env.example .env    # fill in DB_* and UPSTREAM_BASE_URL
npm install
npm run dev              # nodemon, restarts on file changes
```

Service available at `http://localhost:3001`. Requires a local MySQL instance with the `emis_alerts` database created. Migrations run automatically on startup using `DB_MIGRATE_USER`.

18.2 Docker (local stack)

Added to `infrastructure/docker-swarm/docker-compose.local.yml`:

- `alert-service` container (Node.js app, non-root user)
- `alert-mysql` container (dedicated MySQL 8 instance)

Both join the existing Docker overlay network.

18.3 Production

Added to `infrastructure/docker-swarm/docker-stack.yml`. The production MySQL for the alert service is a **separate instance** from the main EMIS MySQL — enforcing DB-level isolation between services. The `alert-service` container runs as UID 1000 (non-root).

18.4 Pre-production security checklist

`NODE_ENV=production` is set

`DB_USER` is a restricted MySQL user (no `DROP`, no `GRANT`)

`CORS_ORIGIN` is the exact production frontend URL — no wildcard

`WS02_JWKS_URI` and `WS02_ISSUER` point to the production IS instance

TLS termination is in place at the nginx-proxy or load balancer layer

`npm audit` returns no high or critical vulnerabilities

Docker image built with multi-stage Dockerfile (not dev mode)

`.env` file is not committed and not baked into the Docker image

`DB_MIGRATE_USER` credentials are rotated after initial migration

19 Summary

	Before	After
Alert data lives in	Laravel's shared database	Dedicated MongoDB alert service
Database engine	MySQL (shared)	MongoDB Community (self-hosted, no overseas)
Phase 2 event queue	N/A	Redis Streams (self-hosted)
Frontend talks to	Laravel backend directly	Alert microservice
JWT validation	Upstream only	Alert service + upstream (defense in depth)
Rate limiting	None for alert traffic	Three-tier per-IP limiting
Future services publish alerts	No — must write to Laravel DB	Yes — API key to alert service
Alert API consistency	3 patterns across 5 files	One file, one contract
Notification foundation	Not present	In place from day one
APIM readiness	N/A	One env var change

End of proposal. Branch: `be/feat/alert-service`